



LensHH-LT · MeritEvalBench

Merit-Function Throughput Across CPUs and GPUs

Eight systems · three lenses · one bit-identical kernel

June 2026

Eight systems, one binary

From a fanless laptop to an H100 — the same MeritEvalBench, the same engine.

System	CPU	GPU	GPU arch (SMs)	Mem BW
Laptop	Core i7-1255U (12T)	— (Iris Xe)	—	—
MacBook Air	Apple Silicon (ARM64)	— (integrated)	—	—
Mini-desktop	Ryzen 3 5300G (8T)	— (Radeon)	—	—
Desktop	Core i5-13400F (16T)	RTX 4060	Ada sm_89 (24)	272 GB/s
Workstation	Ryzen 7 9800X3D (16T)	RTX 5070 Ti	Blackwell sm_120 (70)	~896 GB/s
Cloud	16 vCPU	A100 40GB	Ampere sm_80 (108)	~1.55 TB/s
Cloud	24 vCPU	H100 80GB	Hopper sm_90 (132)	~3.35 TB/s
Cloud	28 vCPU	RTX PRO 6000	Blackwell sm_120 (188)	~1.79 TB/s

Linux / Windows / macOS · GPU via prebuilt native cubin (sm_80/89/90/120) — no CUDA toolkit on the target machine.

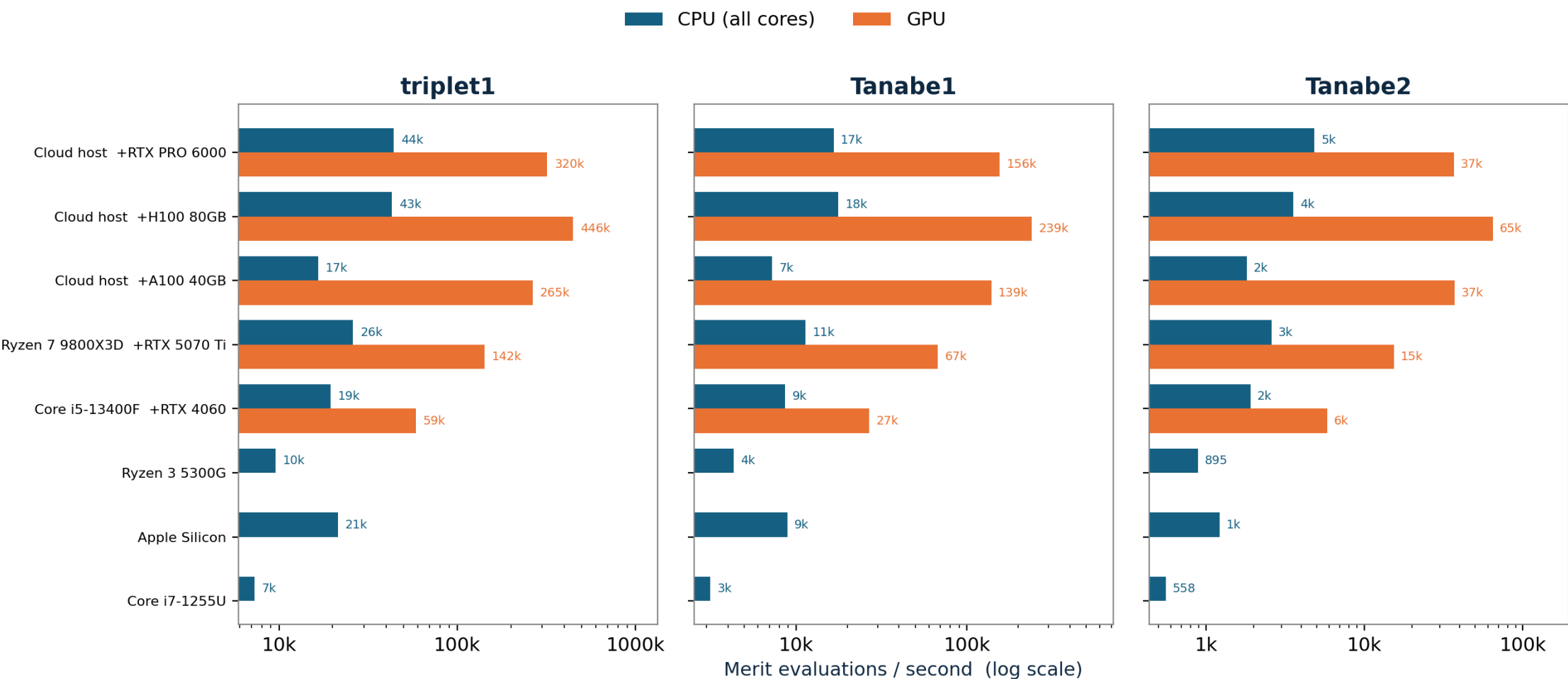
Three test lenses

Small to large merit functions — the GPU advantage grows with merit size.

- ▶ **triplet1** — Cooke triplet, 8 surfaces, 12 variables. Light merit: 7 operands → 240 expanded.
- ▶ **Tanabe1** — imaging lens (US 12,321,041 B2, Example 2), 21 surfaces, 36 variables. WAVEX wavefront-error operand on Forbes quadrature, 6 rings × 12 arms: 8 operands → 241 expanded.
- ▶ **Tanabe2** — same lens and WAVEX, plus a SENS sensitivity operand for manufacturability. SENS perturbs every surface, expanding the merit to 8 operands → 4,686 sub-operands — ~19× more work per evaluation.
- ▶ **All three** — 3 wavelengths, 3 fields, infinite conjugate; the identical merit runs on every system.

Merit evaluations per second — CPU vs GPU

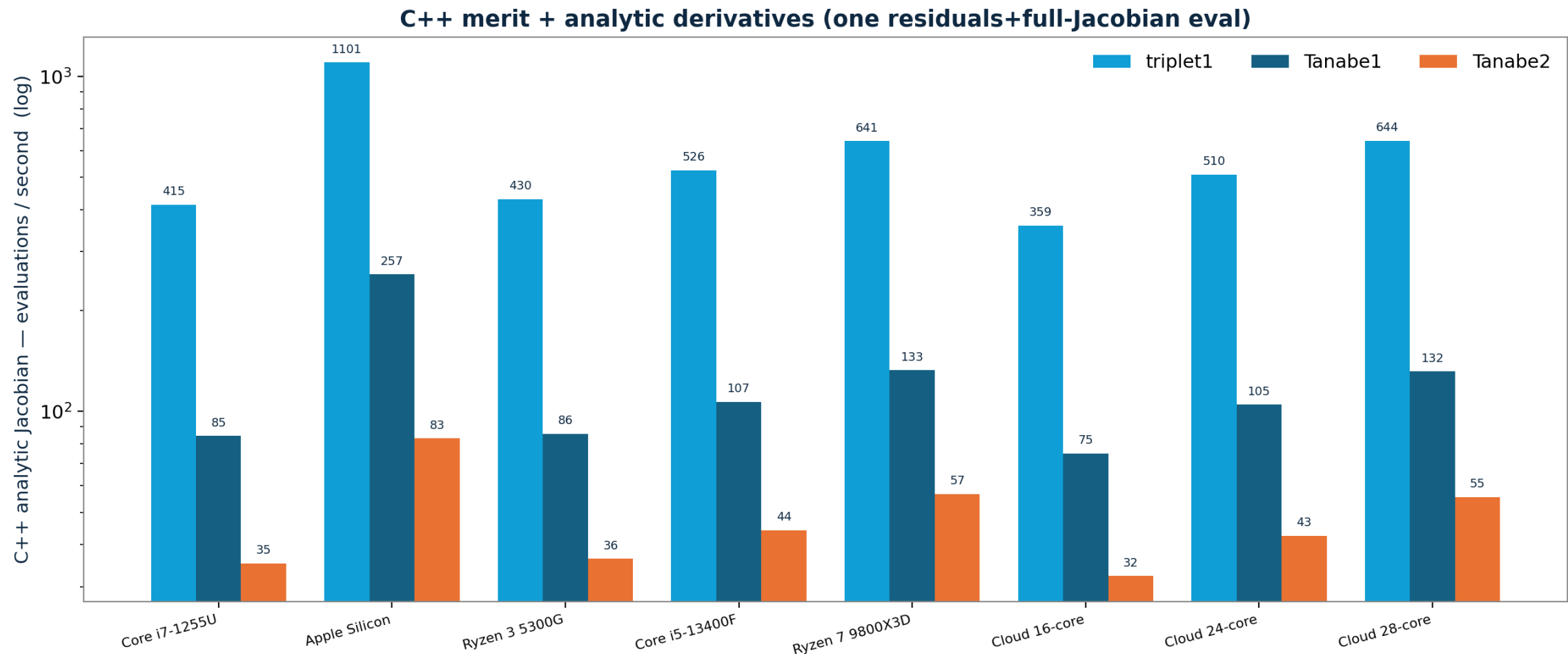
Every lens is a GPU thread; the device evaluates thousands of designs at once.



CPU = best all-core throughput (C# / C++). GPU = whole-merit value kernel at device-fill. GPU output bit-identical to CPU ($|GPU-CPU| \leq 6.6e-14$) on every card.

C++ merit + analytic derivatives

Residuals and the full Jacobian in one pass — the CPU optimizer's hot path.



One evaluation = residuals + full Jacobian via forward-mode dual numbers (analytic). Runs on the CPU; the GPU path is value-only.

What the numbers say

One mental model, portable everywhere.

- ▶ **Bit-identical everywhere** — the same kernel runs on laptop, desktop, Apple Silicon and datacenter; CPU and GPU agree to the FP64 noise floor.
- ▶ **Bandwidth-bound, not FP64-bound** — consumer Blackwell $\approx$ A100 despite $\sim 14\times$ less FP64; throughput tracks memory bandwidth.
- ▶ **The GPU win grows with merit size** — exactly where real lens-design optimization spends its time.
- ▶ **Native cubin, instant launch** — sm_80/89/90/120 prebuilt; the RTX 5070 Ti ran with no SDK installed, no first-run JIT.
- ▶ **C++ analytic Jacobian** — residuals + full Jacobian in one pass keeps the CPU LM polish fast across every machine.